

C

Some MATLAB[®] code and examples

C.1 Transmitter

MATLAB code 1 OTFS frame parameters.

```
% number of Doppler bins (time slots)
N=16;

% number of delay bins (subcarriers)
M=64;

% normalized DFT matrix
Fn=dftmtx(N);
Fn=Fn/norm(Fn);

% subcarrier spacing
delta_f=15e3;

% block duration
T=1/delta_f;

% carrier frequency
fc=4e9;

% speed of light
c=299792458;

% OTFS grid delay and Doppler resolution
delay_resolution = 1/(M*delta_f);
Doppler_resolution = 1/(N*T);
```

MATLAB code 2 Generate OTFS frame.

```

%modulation size
mod_size=4;

% number of information symbols in one frame
N_syms_per_frame=N*M;

% number of information bits in one frame
N_bits_per_frame=N*M*log2(mod_size);

% generate random bits
tx_info_bits=randi([0,1],N_bits_per_frame,1);

% QAM modulation
tx_info_symbols=qammod(tx_info_bits,mod_size,'gray','InputType','bit');

% Generate the MxN OTFS delay-Doppler frame
X=reshape(tx_info_symbols,M,N);

% Vectorized OTFS frame information symbols
x=reshape(X.',N*M,1);

```

MATLAB code 3 OTFS modulation.

```

Im=eye(M);

% row-column permutation matrix (Eq. (4.33))
P=zeros(N*M,N*M);
for j=1:N
    for i=1:M
        E=zeros(M,N);
        E(i,j)=1;
        P((j-1)*M+1:j*M,(i-1)*N+1:i*N)=E;
    end
end

% Method 1 (Eqs. (4.19) and (4.20))
X_tilda=X*Fn';
s=reshape(X_tilda,1,N*M);

% Method 2 (Eq. (4.35))
s=P*kron(Im,Fn')*x;

% Method 3 (Eq. (4.35))
s=kron(Fn',Im)*P*x;

```

C.2 Channel

MATLAB code 4 Channel mobility parameters.

```
% maximum user equipment speed
max_UE_speed_kmh=100;
max_UE_speed = max_UE_speed_kmh*(1000/3600);

% maximum Doppler spread (one-sided)
nu_max = (max_UE_speed*fc)/(c);

% maximum normalized Doppler spread (one-sided)
k_max = nu_max/Doppler_resolution;
```

MATLAB code 5 3GPP standard channels (from Section 2.5.1).

```
% EPA model
delays_EPA = [0, 30, 70, 90, 110, 190, 410]*10-9;
pdp_EPA = [0.0, -1.0, -2.0, -3.0, -8.0, -17.2, -20.8];

% EVA model
delays_EVA = [0, 30, 150, 310, 370, 710, 1090, 1730, 2510]*10-9;
pdp_EVA = [0.0, -1.5, -1.4, -3.6, -0.6, -9.1, -7.0, -12.0, -16.9];

% ETU model
delays_ETU = [0, 50, 120, 200, 230, 500, 1600, 2300, 5000]*10-9;
pdp_ETU = [-1.0, -1.0, -1.0, 0.0, 0.0, 0.0, -3.0, -5.0, -7.0];
```

MATLAB code 6 Generate standard channel parameters.

```
% choose channel model, for example: EVA
delays=delays_EVA;
pdp=pdp_EVA;
% dB to linear scale
pdp_linear = 10.^(pdp/10);
% normalization
pdp_linear = pdp_linear/sum(pdp_linear);
% number of propagation paths (taps)
taps=length(pdp);

% generate channel coefficients (Rayleigh fading)
g_i = sqrt(pdp_linear).*(sqrt(1/2) * (randn(1,taps)+1i*randn(1,taps)));

% generate delay taps (assuming integer delay taps)
l_i=round(delays./delay_resolution);
```

```
% Generate Doppler taps (assuming Jakes spectrum)
k_i = (k_max*cos(2*pi*rand(1,taps)));
```

MATLAB code 7 Generate synthetic channel parameters.

```
% number of propagation paths
taps=6;

% maximum normalized delay and Doppler spread
l_max=4;
k_max=4;

% generate channel coefficients (Rayleigh fading) with uniform pdp
g_i = sqrt(1/taps).*(sqrt(1/2) * (randn(1,taps)+1i*randn(1,taps)));

% generate delay taps uniformly from [0,l_max]
l_i = [randi([0,l_max],1,taps)];
l_i = l_i-min(l_i);

% generate Doppler taps (assuming uniform spectrum [-k_max,k_max])
k_i = k_max-2*k_max*rand(1,taps);
```

MATLAB code 8 Generate discrete delay-time channel coefficients and matrix.

```
z=exp(1i*2*pi/N/M);
delay_spread=max(l_i);

% Generate discrete-time baseband channel in TDL form (Eq. (2.22))
gs=zeros(delay_spread+1,N*M);
for q=0:N*M-1
    for i=1:taps
        gs(l_i(i)+1,q+1)=gs(l_i(i)+1,q+1)+g_i(i)*z^(k_i(i)*(q-l_i(i)));
    end
end

% Generate discrete-time baseband channel matrix (Eq. (4.38))
G=zeros(N*M,N*M);
for q=0:N*M-1
    for ell=0:delay_spread
        if(q>=ell)
            G(q+1,q-ell+1)=gs(ell+1,q+1);
        end
    end
end
```

MATLAB code 9 Generate delay-time and delay-Doppler channel matrix.

```
% generate delay-time channel matrix (Eq. (4.55))
H_tilda=P*G*P';

% generate delay-Doppler channel matrix (Eq. (6.1))
H=kron(Im,Fn)*(P'*G*P)*kron(Im,Fn');
```

MATLAB code 10 Generate r by passing the Tx signal through the channel.

```
% Method 1: Using the TDL model (Eq. (4.36))
r=zeros(N*M,1);
for q=0:N*M-1
    for ell=0:(delay_spread-1)
        if(q>=ell)
            r(q+1)=r(q+1)+gs(ell+1,q+1)*s(q-ell+1);
        end
    end
end

% Method 2: Using the time-domain channel matrix (G) (Eq. (4.37))
r=G*s.';

% Method 3: Using the delay-time channel matrix (H_tilda) (Eq. (4.54))
x_tilda=reshape(X_tilda.',N*M,1);
y_tilda=H_tilda*x_tilda;
r=P*y_tilda;

% Method 4: Using the delay-Doppler channel matrix (H) (Eq. (4.59))
x=reshape(X.',N*M,1);
y=H*x;
r=P*kron(Im,Fn')*y;
```

C.3 Receiver

MATLAB code 11 Add AWGN.

```
% average QAM symbol energy
Es = mean(abs(qammod(0:mod_size-1,mod_size).^ 2));

% SNR=Es/noise power
SNR_dB = 25;
SNR=10.^(SNR_dB/10)

% noise power
sigma_w_2=Es/SNR
```

```
% generate Gaussian noise samples with variance=sigma_w_2
noise = sqrt(sigma_w_2/2)*(randn(N*M,1) + 1i*randn(N*M,1));

% add AWGN to the received signal
r=r+noise;
```

MATLAB code 12 OTFS demodulation.

```
% Method 1 (Eqs. (4.24) and (4.27))
Y_tilda=reshape(r,M,N);
Y=Y_tilda*Fn;

% Method 2 (Eq. (4.35))
y=kron(eye(M),Fn)*(P.)*r;
Y=reshape(y,N,M).';

% Method 3 (Eq. (4.35))
y=(P.)*kron(Fn,eye(M))*r;
Y=reshape(y,N,M).';
```

MATLAB code 13 OTFS delay-Doppler LMMSE detection.

```
% vectorize Y
y=reshape(Y.',N*M,1);

% estimated delay-Doppler matrix (Eq. (6.18))
x_hat=(H'*H+sigma_w_2)^(-1)*(H'*y);

% QAM demodulation
x_hat=qamdemod(x_hat,mod_size,'gray');
```

MATLAB code 14 OTFS time domain LMMSE detection.

```
% estimated time domain samples (Eq. (6.19))
s_hat=(G'*G+sigma_w_2)^(-1)*(G'*r);

% MxN estimated delay-Doppler symbols (using Method 1 in code 12)
X_hat_tilda=reshape(s_hat,M,N);
X_hat=X_hat_tilda*Fn;
x_hat=reshape(X_hat.',N*M,1);

% QAM demodulation
x_hat=qamdemod(x_hat,mod_size,'gray');
```

C.4 Generate G matrix and received signal for OTFS variants

MATLAB code 15 RZP-OTFS.

```

z=exp(1i*2*pi/N/M);
delay_spread=max(l_i);

% Generate discrete-time baseband channel in TDL form (Eq. (2.22))
gs=zeros(delay_spread+1,N*M);
for q=0:N*M-1
    for i=1:taps
        gs(l_i(i)+1,q+1)=gs(l_i(i)+1,q+1)+g_i(i)*z^(k_i(i)*(q-l_i(i)));
    end
end

% Generate discrete-time baseband channel matrix (Eq. (4.38))
G_rzp=zeros(N*M,N*M);
for q=0:N*M-1
    for ell=0:delay_spread
        if(q>=ell)
            G_rzp(q+1,q-ell+1)=gs(ell+1,q+1);
        end
    end
end

% generate received signal after discarding CP
r=G_rzp*s.';

```

MATLAB code 16 RCP-OTFS.

```

z=exp(1i*2*pi/N/M);
delay_spread=max(l_i);

% Generate discrete-time baseband channel in TDL form (Eq. (2.22))
gs=zeros(delay_spread+1,N*M);
for q=0:N*M-1
    for i=1:taps
        gs(l_i(i)+1,q+1)=gs(l_i(i)+1,q+1)+g_i(i)*z^(k_i(i)*(q-l_i(i)));
    end
end

% Generate discrete-time baseband channel matrix (Eq. (4.83))
G_rcp=zeros(N*M,N*M);
for q=0:N*M-1
    for ell=0:delay_spread
        G_rcp(q+1,mod(q-ell,N*M)+1)=gs(ell+1,q+1);
    end
end

```

```
% generate received signal after discarding CP
r=G_rcp*s.';
```

MATLAB code 17 CP-OTFS.

```
z=exp(1i*2*pi/N/M);
delay_spread=max(l_i);
l_cp=delay_spread;

% Generate discrete-time baseband channel in TDL form (Eq. (2.22))
gs=zeros(delay_spread+1,N*(M+l_cp));
for q=0:N*(M+l_cp)-1
    for i=1:taps
        gs(l_i(i)+1,q+1)=gs(l_i(i)+1,q+1)+g_i(i)*z^(k_i(i)*(q-l_i(i)));
    end
end

% Generate discrete-time baseband channel matrix (Eq. (4.93))
G_cp=zeros(N*M,N*M);
for n=0:N-1
    for m=0:M-1
        for ell=0:delay_spread
            G_cp(m+n*M+1,n*M+mod(m-ell,M)+1)=gs(ell+1,m+n*M+l_cp+1);
        end
    end
end

% generate received signal after discarding CP per block
r=G_cp*s.';
```

MATLAB code 18 ZP-OTFS.

```
z=exp(1i*2*pi/N/M);
delay_spread=max(l_i);
l_zp=delay_spread;

% Generate discrete-time baseband channel in TDL form (Eq. (2.22))
gs=zeros(delay_spread+1,N*(M+l_zp));
for q=0:N*(M+l_zp)-1
    for i=1:taps
        gs(l_i(i)+1,q+1)=gs(l_i(i)+1,q+1)+g_i(i)*z^(k_i(i)*(q-l_i(i)));
    end
end

% Generate discrete-time baseband channel matrix (Eq. (4.109))
G_zp=zeros(N*M,N*M);
for n=0:N-1
```



```
for m=0:M-1
    for ell=0:delay_spread
        if(m>=ell)
            G_zp(m+n*M+1,m+n*M-ell+1)=gs(ell+1,m+n*M+l_zp+1);
        end
    end
end
end

% generate received signal after discarding ZP per block
r=G_zp*s.';
```